Hibernate Many-to-Many Mapping — Scenario-Based Tasks

Task 1: Space Mission and Astronaut Management System

Scenario: A space research organization manages multiple space missions. Each mission can include multiple astronauts, and each astronaut can participate in multiple missions.

Entities:

- **SpaceMission** — missionId, missionName, destination, launchYear
- **Astronaut** — astronautId, astronautName, specialization, experienceYears

Requirements:

1. Create both entity classes using Hibernate annotations.
2. Implement a bidirectional `@ManyToMany` relationship.
3. Use `@JoinTable` with the table name `mission_astronaut`.
4. Store mission and astronaut details in the database.
5. Retrieve all astronauts assigned to a particular mission.
6. Retrieve all missions in which a particular astronaut has participated.
7. Remove an astronaut from a mission without deleting the astronaut record.
8. Display the complete mission and astronaut details.

Task 2: Wildlife Conservation and Researcher Management

Scenario: A wildlife conservation organization conducts multiple research projects. Each project involves several researchers, and researchers can work on multiple projects.

Entities:

- **ConservationProject** — projectId, projectName, forestRegion, budget
- **WildlifeResearcher** — researcherId, researcherName, expertise, qualification

Requirements:

1. Create a Many-to-Many relationship between projects and researchers.
2. Use a join table named `project_researcher`.
3. Save projects with multiple researchers.
4. Find all researchers working in a particular forest region's projects.
5. Find all projects assigned to a researcher.
6. Add a new researcher to an existing project.
7. Remove a project association without deleting the researcher.
8. Implement bidirectional navigation.

Common technical instructions for all tasks

- Use Java, Hibernate ORM, and MySQL.
- Use annotation-based entity mapping.

- Apply `@Entity`, `@Id`, `@GeneratedValue`, `@ManyToMany`, and `@JoinTable` where appropriate.
- Configure the owning and inverse sides using `mappedBy` for bidirectional mapping.
- Write a main class to test each operation.
- Avoid infinite recursion when printing bidirectional entity relationships.
- Do not use Spring Data JPA; implement the tasks using Hibernate `SessionFactory` and `Session`.